HookSteg: A Truly Lossless Steganography Based on Process Hooking for Lossy Network Channels

Mingliang Zhang, Yi Zhang, Yanmei Liu, Wentong Fan, and Xiangyang Luo

***Abstract*—Covert communication often relies on image steganography to transmit sensitive data such as commercial secrets. Any minor bit error can lead to catastrophic consequences, which imposes extremely high demands on the zero-error transmission of secret information. However, existing robust image steganographic schemes still fail to guarantee zero-error recovery of secret bits when resisting hybrid attacks such as compression and scaling on social platforms. To address this issue, we propose a lossless steganographic method called HookSteg, which bypasses lossy attacks on social platforms through process hooking, thereby achieving lossless steganography. This method first identifies nodes affected by lossy attacks through tracking whether image data has been compromised. Subsequently, a method for capturing lossy attack instructions is proposed based on syntax-semantic joint features and dynamic taint tracking techniques. Finally, process hooking is utilized to bypass lossy attack instructions, ensuring the lossless transmission of secret information. Experimental results on transmission scenarios with tens of millions of daily active users, such as Youku and Kutumb, demonstrate that HookSteg is general and requires no modifications to existing steganographic algorithms. It achieves, for the first time, lossless transmission of secret information in real-world scenarios, with an average improvement in extraction accuracy of 36.63%. Additionally, its efficiency in locating attack instructions is improved by 94.34% compared to traditional manual reverse engineering methods. Furthermore, this study resolves the trade-off among embedding capacity, security, and robustness, enabling steganography to focus on enhancing security and capacity, thereby providing a general and efficient solution for lossless steganography.**

***Index Terms*—Covert communication, image steganography, lossy channel, lossless steganography, robustness.**

I. Introduction

IN TODAY's information age, the internet and social media have deeply penetrated every aspect of social life, greatly expanding the channels for the public to access and spread information. Alongside the convenience of communication comes the persistent challenge to privacy and security. Even when encryption is employed, both communication content and metadata remain susceptible to analysis, identification, and potential leakage [1], [2]. Furthermore, the act of communication itself is visible, and its content, even if encrypted, cannot escape the possibility of surveillance and censorship. Therefore, true communication security depends not only on the confidentiality of content but also on the invisibility of the communication act itself.

Against this backdrop, image steganography [3], as a technique for hiding information within images, offers a viable solution. This technology exploits the statistical redundancies of human vision and digital media to hide secret information into public images for transmission over open networks [4]. Specifically, the cover image [5] refers to the original image used to conceal information, while the stego image [6] denotes the image after secret data has been hidden. Steganographic techniques not only conceal the communication content but also mask the "act of secret communication" itself, serving as an effective means of transmitting private information in public environments [7].

Unfortunately, social platforms typically apply compression, scaling, or watermarking to uploaded images to optimize bandwidth and user experience [8]. These operations often damage stego images, leading to extraction failure and severely limiting the practical application of steganography [9]. This demands that steganographic algorithms possess strong robustness to resist various unintentional or malicious processing [10].

Most existing research [9], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28] focuses on identifying robust regions or features within images and uses error-correcting codes [29] to embed secret information into them to achieve robust steganography. However, when confronted with the complex and varied mixed attacks present in real-world scenarios, such methods often fail to ensure the complete extraction of the hidden information, and thus cannot be considered truly robust steganographic solutions. In sensitive data transmission scenarios with extremely high requirements for information integrity, true lossless steganography is urgently needed under lossy channels.

To address this, we propose a lossless steganographic method based on process hooking, serving as a general and efficient solution. It can provide truly lossless recovery capabilities in social platforms. The distinction between the proposed method and existing methods is illustrated in Fig. 1. By modifying the image processing logic on the client side through process hooking, our method cleverly bypasses the lossy attacks, thereby achieving lossless transmission of the secret information.

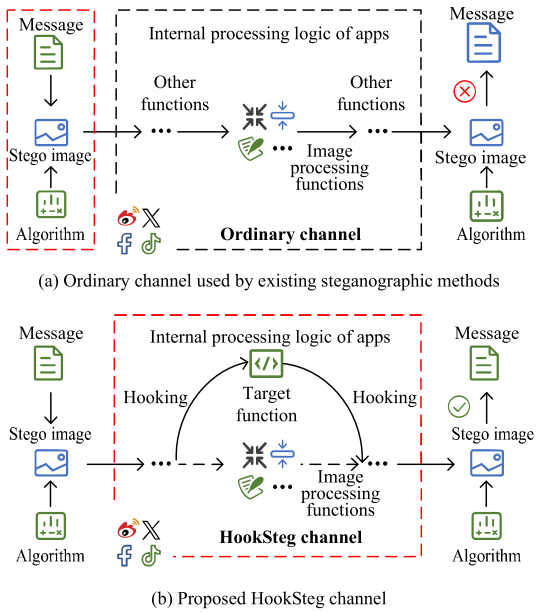
Received 10 April 2026; revised 14 June 2026; accepted 24 June 2026. Date of publication 26 June 2026; date of current version 2 July 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U23A20305, in part by the Natural Science Foundation for Excellent Young Scholars of Henan Province under Grant 252300421233, and in part by the Innovation Scientists and Technicians Troop Construction Projects of Henan Province under Grant 254000510007. The associate editor coordinating the review of this article and approving it for publication was Dr. Baoyuan Wu. *(Corresponding author: Xiangyang Luo.)*

The authors are with the Information Engineering University, Zhengzhou 450001, China, and also with Henan Key Laboratory of Cyberspace Situation Awareness, Zhengzhou 450001, China (e-mail: zmlmail2021@163.com; tzyy4001@sina.com; lym129@zua.edu.cn; fftxxgc@sina.com; luoxy_ieu@sina.com).

Digital Object Identifier 10.1109/TIFS.2026.3707784

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.



(a) Ordinary channel used by existing steganographic methods

(b) Proposed HookSteg channel

Fig. 1. The differences between the current works and HookSteg.

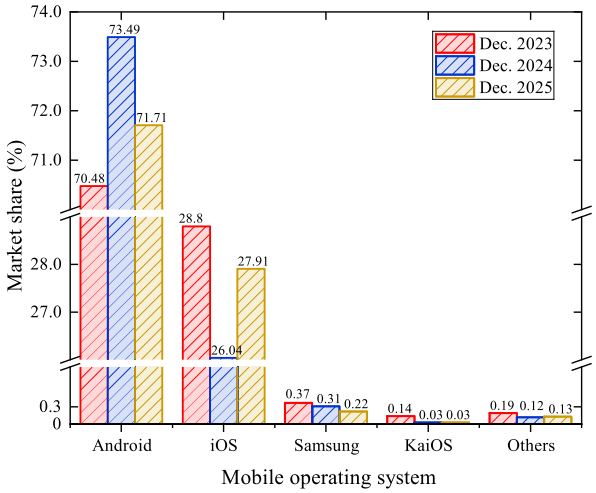


Fig. 2. Market share of mobile device operating systems.

The main contributions of this paper are as follows:

- A steganographic method based on process hooking bypassing lossy attacks is proposed to address the issue of achieving lossless transmission of stego images under hybrid attacks in real-world scenarios. For the first time, this method realizes lossless and reliable information transmission in complex attack environments. The proposed method exhibits strong universal adaptability, improving the average extraction accuracy by 36.63% without requiring any modifications to existing steganographic algorithms, guaranteeing lossless transmission of secret information.
- An attack node identification method for stego images is designed to provide core technical support for subsequent attack instruction tracing and localization. This method innovatively integrates dynamic and static dual-analysis mechanisms to locate the critical communication nodes causing information transmission loss throughout the entire image processing workflow. Experimental validation demonstrates that this method can effectively detect and verify attack nodes in 76.66% of target platforms, filling the gap in locating the sources of attacks against stego images.
- An attack instruction localization algorithm based on syntax-semantic joint features is proposed to efficiently filter and identify attack instructions from large-scale program instruction sets. This algorithm constructs an automated filtering mechanism for decompiled statements, leveraging joint feature constraints from both syntactic and semantic dimensions, combined with dynamic taint tracking technology. It enables rapid identification of attack instructions, directly filtering out 94.34% of non-image-processing irrelevant statements, significantly enhancing the efficiency and precision of attack instruction identification.

The remainder of this paper is organized as follows. Section II presents the research background and related work. In Section III, we provide a comprehensive description of HookSteg. Section IV demonstrates the effectiveness of HookSteg in principle. Section V validates the effectiveness of HookSteg through experiments. Section VI concludes the paper. Finally, Section VII discusses future work.

II. Background and Related Work

This section reviews the research background and related work.

A. Threat Model

Due to the convenience of mobile devices, users often rely on them to access social platforms. The mobile operating system, which includes Android, iOS, Samsung, among others, serves as the core of modern mobile devices. It not only determines the basic functionality of the device but also influences the user experience and application (app) ecosystem. Fig. 2 shows the market share of mobile device operating systems, with data sourced from StatCounter, an authority in traffic statistics and analysis services. It indicates that over the past three years, the Android operating system has maintained a market share of over 70.48%, demonstrating its dominant position in the market. For ease of explanation, this paper discusses and experimentally validates HookSteg on the widely used Android operating system.

This paper considers a threat model involving two parties:

- The sender possesses a mobile device with root privileges and obtains permission to execute process hooking, using this device to upload stego images to a social platform client.
- The receiver, utilizing a conventional mobile device or a web browser, completes legitimate login using the social platform account credentials provided by the sender, thereby enabling the download of stego images.

B. Related Work

This section reviews related work in image steganography, including modification-based image steganography, mapping-based image steganography, and generation-based image steganography.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

1) Modification-Based Image Steganography: Modification-based image steganography embeds secret information by modifying the pixels or coefficients of a cover image [30] and employs specific optimization strategies along with error-correction mechanisms. For instance, the robust tri-stage TLM-DNA coding scheme proposed by Alexan et al. [29] significantly enhances robustness against lossy channel attacks. These ensure that the secret information can still be extracted after being transmitted through lossy channels. Early studies [31] primarily focus on heuristically designed strategies that balance the security and robustness of steganography by selecting and modifying coefficients in transform domains such as the discrete cosine transform (DCT) and discrete wavelet transform (DWT), or pixels in the spatial domain. To address channel attacks such as JPEG recompression that may occur during actual transmission, researchers further introduce dither modulation [32], [33] and adaptive error correction coding [11], [34], aiming to ensure robustness while reducing the magnitude of modifications and enhancing security.

Additionally, some studies [12], [13] improve robustness by recompressing images. In recent years, with the development of deep learning, robust steganographic methods [9], [13] based on deep neural networks have become a research hotspot. Such methods adaptively embed secret information into cover images and introduce techniques such as noise layers and attention mechanisms. This enables the model to automatically learn to hide information in complex texture regions and enhance robustness against various types of noise, thereby improving both security and anti-interference capabilities. Modification-based embedding algorithms offer strong robustness against specific or multiple image processing attacks. However, they inevitably introduce distortion to the cover image, which can lead to the leakage of steganographic traces and fails to ensure lossless transmission of secret information over lossy channels.

2) Mapping-Based Image Steganography: Early mapping-based image steganography draws inspiration from image retrieval [14], [15], establishing a one-to-one correspondence between specific secret information and visual features of each image, such as color histograms [35], SIFT feature points [36], or hash sequences. As the images themselves remain unmodified, these methods can resist steganalysis [37]. However, it is challenging to construct an image database that exhaustively covers all possible information fragments, and feature instability can lead to mapping mismatches. To address these issues, subsequent research [16], [17] introduces the combination of robust hashing [38] and error correction coding, designing perceptual hashing algorithms that are more robust to image processing operations to tolerate minor deviations during feature extraction.

In recent years, with the advancement of deep learning technologies, this field has further evolved into a mapping paradigm based on deep features [18], [19], [20], [21]. This approach utilizes deep neural networks to extract more discriminative and robust mid-level semantic features from images. It then constructs denser, more stable codebooks in high-dimensional feature spaces through clustering or quantization, significantly enhancing robustness against complex channel attacks. Although mapping-based image steganography exhibits strong robustness, its hiding capacity is relatively limited, which to some extent restricts the application potential of this type of method in large-scale data transmission scenarios.

3) Generation-Based Image Steganography: Generation-based image steganographic methods [22], [23], [24], [25], [26], [27], [28] have attracted increasing research attention in recent years. These methods primarily utilize generative models such as Generative Adversarial Networks (GANs) [39] and diffusion models [40], [41], [42] to hide secret information directly during the image generation phase, achieving a shift from modifying the cover to generating the cover. Early research was mainly based on GANs [23], [43], which fit the real data distribution through the dynamic game between a generator and a discriminator, offering high generation efficiency as well as good concealment and robustness. However, these methods still suffer from issues such as unrealistic synthetic images, training instability, and mode collapse [22], [44].

To overcome these limitations, recent studies have begun exploring steganographic methods based on diffusion models [28], [45]. These methods generate high-quality and diverse samples through gradual denoising, avoiding mode collapse and making the secret information more concealed and natural. Nevertheless, diffusion models exhibit numerical instability and sensitivity to perturbations during the generation process [25], resulting in insufficient robustness in the recovery of secret information, especially when facing unknown channel attacks.

III. Proposed Method

To achieve lossless transmission of secret information over lossy channels, we propose an efficient and highly general lossless steganographic method that cleverly circumvents potential interference from apps with stego images by hooking the instructions used for image attacks. Without altering existing steganographic methods, HookSteg provides a genuinely lossless remedy for all schemes that break down in real-world scenarios.

A. Overview of HookSteg

The HookSteg, illustrated in Fig. 3, consists of two main parts: the preprocessing process and the covert communication process. The preprocessing process includes three steps: identify attack nodes, extract suspicious functions, and locate lossy functions. The covert communication process comprises two steps: send secret information and extract secret information.

The core objective of the preprocessing process is to accurately locate the instructions that attack the image. It then identifies the function where these instructions reside, referred to as the lossy functions, to provide support for subsequently bypassing the lossy attacks. This process first verifies whether the stego image is only attacked on the app's client side. If the stego image is only attacked on the client side, it utilizes syntax-semantic joint features to filter relevant image processing instructions. Subsequently, based on the obtained instructions, it extracts suspicious statements with the aid of image processing features. Then, combined with dynamic taint tracking technology, it hooks the process of the suspicious

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

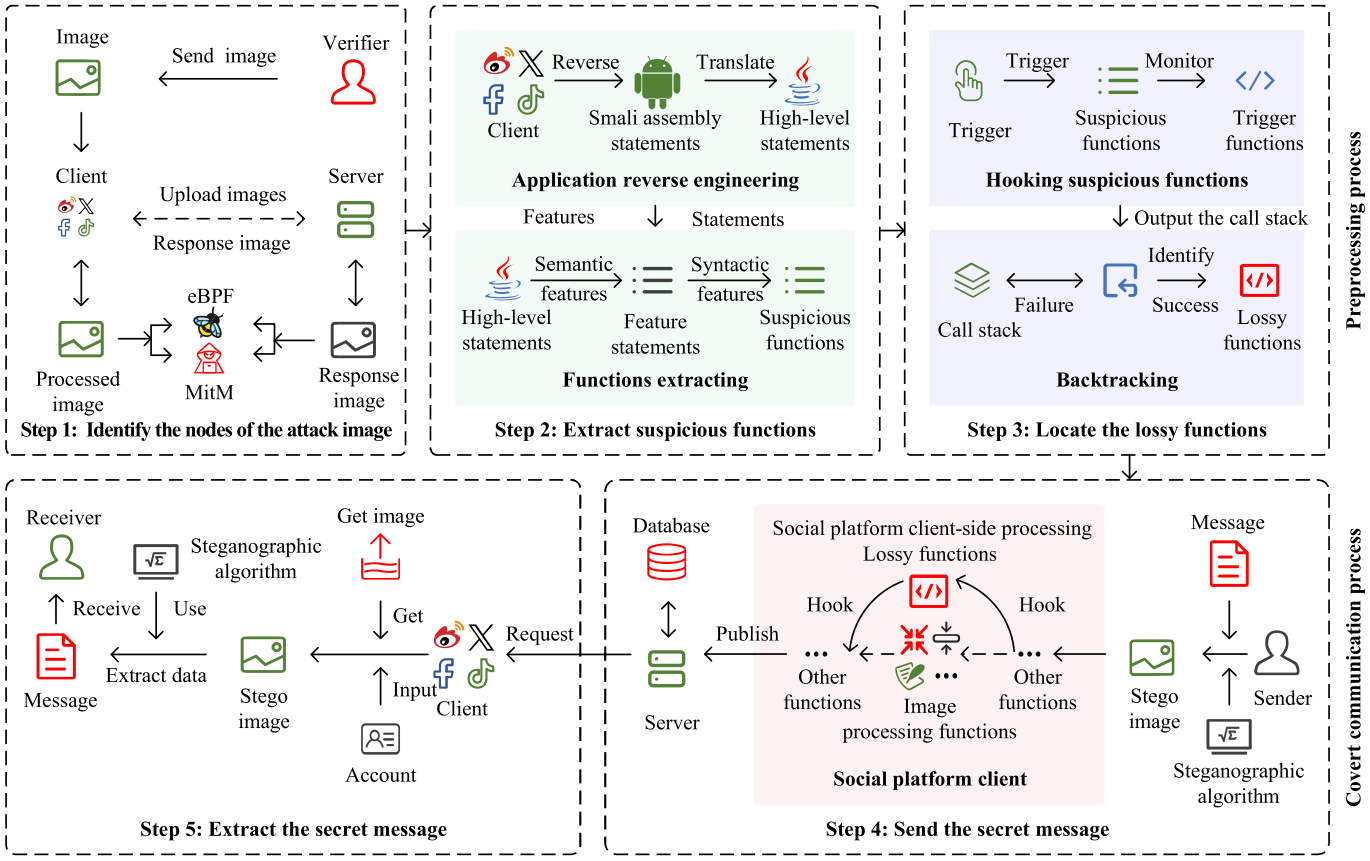


Fig. 3. The diagram of HookSteg.

function to be verified on the client side and sends the stego image to trigger this function. Finally, an attempt is made to modify the triggered function. If the function can be successfully modified so that it no longer attacks the stego image, it is identified as the lossy functions. If the function cannot be modified, the next function from the call stack is retrieved and tested, repeating the process until the lossy functions are successfully identified.

In covert communication, the core task for the sender is to ensure that the stego image retains the secret information intact even after being processed by social platforms. At this stage, the lossy functions identified during the preprocessing phase play a critical role. The sender will intercept the call to the lossy function using hooking technology and bypass this function in a zero-copy manner, thereby avoiding attacks on the image. For the receiver, the process is relatively straightforward and does not require any complex configuration. The receiver only needs to analyze the obtained image using a pre-agreed extraction algorithm, just like an ordinary user, to accurately and completely restore the information transmitted by the sender.

B. Identify Attack Nodes

For different social media platforms, the nodes (server-side, client-side) and methods of image processing vary significantly. Some platforms perform processing only at the client-side node, whereas others apply it at both client and server nodes. The operations themselves also differ ranging from compression and rescaling to watermark injection.

In order to verify whether a specific app can establish a HookSteg lossless channel, we need to analyze and identify the execution nodes of lossy functions to provide support for the subsequent construction of the lossless channel. HookSteg employs a dual validation approach primarily based on Man-in-the-Middle (MitM) attacks, supplemented by extended Berkeley Packet Filter (eBPF) technology. The schematic diagram is shown in Fig. 4.

Let C denote the client, S the server, and M the MitM node. $\mathrm{Req}_{C\to S}$ represents the connection request sent from C to S. $\boldsymbol{I}_s$ is the stego image generated by the sender. $\boldsymbol{I}'_s$ denotes the image after client-side processing, and $\boldsymbol{I}''_s$ denotes the final image downloaded from the platform.

The original communication link is:

$$C \leftrightarrow S. \tag{1}$$

After the MitM attacks, it becomes:

$$C \leftrightarrow M \leftrightarrow S. \tag{2}$$

The process for identifying the nodes of the attack image via MitM attacks consists of the following five steps:

❶ Connection establishment request: C sends a connection request $\mathrm{Req}_{C\to S}$ to S.

❷ MitM interception: M intercepts $\mathrm{Req}_{C\to S}$, forges a response to make C believe it has connected to S, while simultaneously M initiates a connection to S, establishing the $M \leftrightarrow S$ link.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

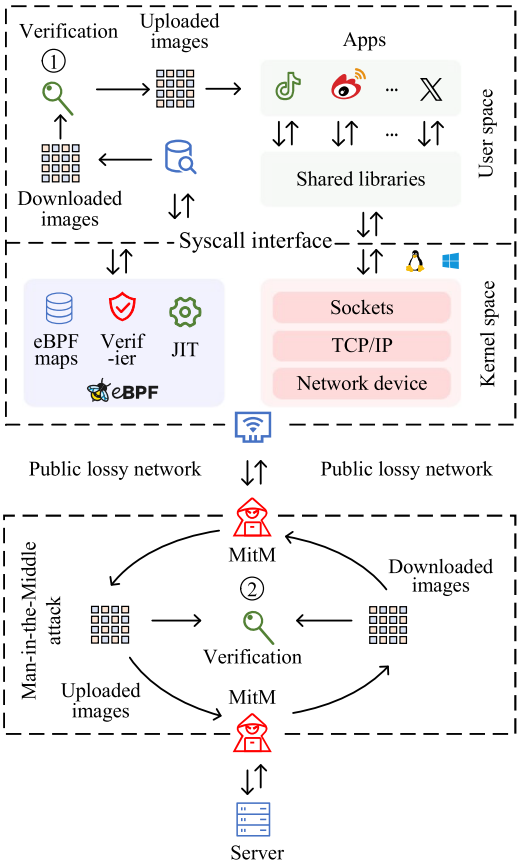


Fig. 4. Schematic diagram of identifying attack nodes.

❸ Image data upload: C processes the stego image $\boldsymbol{I}_s$ through operations such as compression and scaling to generate a processed image $\boldsymbol{I}'_s$, and sends it to S.

❹ Image interception and storage: M intercepts $\boldsymbol{I}'_s$, can store it, and forwards it to S to maintain communication.

❺ Determination of the attack nodes: Download the final image $\boldsymbol{I}''_s$ from the platform and compare the pixel matrices of $\boldsymbol{I}'_s$ and $\boldsymbol{I}''_s$. If $\boldsymbol{I}'_s \equiv \boldsymbol{I}''_s$, it indicates client-side-only processing, enabling the establishment of a HookSteg lossless channel. If $\boldsymbol{I}'_s \not\equiv \boldsymbol{I}''_s$, it signifies server-side participation in processing, preventing the establishment of a HookSteg lossless channel.

When MitM attacks are not feasible, eBPF is used to capture client-side local image data. Let the client process be P_c and the server process be P_s.

❶ eBPF program loading: Write an eBPF program E, load it into the kernel, and monitor P_c's network communication.

❷ Data packet filtering: E filters packets according to rules (such as destination port and protocol type) and captures only the traffic from $P_c \rightarrow P_s$.

❸ Image data extraction: When P_c sends image data $\boldsymbol{I}_s$, E extracts image data blocks $B_1, B_2, \ldots, B_n$ from the network buffer and reassembles them into the complete image $\boldsymbol{I}_s'$.

❹ Data storage: Write $\boldsymbol{I}_s'$ to storage media for subsequent comparison.

❺ Determination of the attack nodes: Similarly, by comparing the pixel matrices of $\boldsymbol{I}'_s$ and the downloaded image $\boldsymbol{I}_s''$, determine whether the processing mechanism involves the server side.

TABLE I
COMMON IMAGE PROCESSING FEATURE CODES

No.	Image processing type	Operation feature code
1	Compression	"compress"
2	Rotation	"rotate"
3	Scaling	"resize"
4	Upload	"upload"

C. Extract Suspicious Functions

Although traditional methods of locating a function through static analysis and dynamic debugging are effective, they also have some obvious drawbacks. Manual analysis of the entire codebase is required, which is particularly challenging for large, complex apps where decompiled code may be difficult to comprehend and potentially obfuscated or encrypted, significantly increasing the analysis difficulty. To rapidly locate lossy functions, we propose a localization method that leverages syntax-semantic joint features.

We define three types of statements: (i) high-level statements, which are obtained through application reverse engineering and further converted into a high-level language; (ii) feature statements, which are selected from the complete set of high-level statements based on semantic features; and (iii) suspicious statements, which are further selected from the complete set of feature statements based on syntactic features.

This stage can be divided into two sub-steps: Search for High-Level Statements (SHS) and Extract Suspicious Statements (ESS), formally represented as:

$$S_v = \mathrm{SHS}(S_h, P), \tag{3}$$

$$S_c = \mathrm{ESS}(S_v, R), \tag{4}$$

where S_h, S_v, and S_c are the sets of high-level statements, feature statements, and suspicious statements, respectively, and the elements in these sets are denoted as s_h, s_v, and s_c, respectively. S_v is the set preliminarily filtered by the semantic features of image processing, and S_c is a set further extracted from S_v containing target statements. P represents the set of semantic features for image processing, and R represents the set of syntactic features for image processing. Their elements are denoted by p and r respectively. Let $\mathrm{SHS}(S_h, P)$ denote the filtering process that selects statements from S_h based on P, yielding S_v. Let $\mathrm{ESS}(S_v, R)$ denote the further filtering process that selects statements from S_v based on R, yielding S_c.

In the SHS stage, the binary app needs to be reverse-engineered into assembly language and then further converted into a high-level language. Based on the semantic feature codes of image processing (as shown in Table I), the feature statement set is obtained. This process can be formally expressed as:

$$S_v = \{s_h \in S_h \mid \exists p \in P, p \in S_h\}. \tag{5}$$

To further narrow the filtering scope, the statements are segmented according to their syntactic features, as illustrated in Fig. 5.

The feature statement set S_v is further extracted by R, and these statements are added to the suspicious statements set S_c, which can be formally expressed as:

$$S_c = \{s_v \mid \exists s_v \in S_v, r \in R\}. \tag{6}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

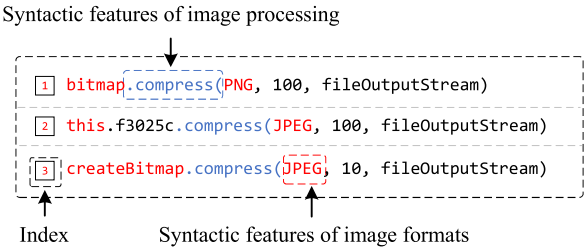


Fig. 5. Statement format.

TABLE II
STRATEGY OF HOOKING FOR BYPASSING ATTACKS ON STEGO IMAGES

No.	Hooking strategy	Implementation method
1	Modify image processing logic	Replace method parameters
2	Modify image processing logic	Intercept method execution
3	Modify image processing logic	Construct return object
4	Modify image upload logic	Replace method parameters

We designate any function that contains suspicious statements as a suspicious functions. Extracting such functions provides crucial support for locating lossy functions.

D. Locating Lossy Functions

To locate the lossy functions, we adopt a dynamic trigger mechanism based on hooking technology and combine it with call stack backtracking for analysis. At the same time, dynamic taint tracking technology is introduced to trace the source of suspicious functions within the API call sequence during image processing.

The interception operation implemented by the hooking technique is denoted by Hook(·). During the app's image upload operation, the function s_c^i corresponding to the i-th statement in $S_c = \{s_c^0, s_c^1, \ldots, s_c^i, \ldots\}$ is monitored via Hook(·). When a certain function s_c^i (referred to as the triggered function) is invoked, its input parameters, return value, and its corresponding call stack S_i are recorded. The call stack $S_i = \{s_i^0, s_i^1, \ldots, s_i^{n-1}\}$ has the function directly manipulating the image, s_i^0, at the top of the stack, while the remaining $s_i^1, s_i^2, \ldots$ are downstream functions within the call stack.

To bypass attacks on the stego image, we intervene in the triggered function or the functions in its call stack according to the hooking strategies L listed in Table II. Specifically, the hooking of the image processing process is achieved by modifying the image processing logic (e.g., replacing method parameters, constructing return objects, intercepting method execution) and modifying the image upload logic (e.g., replacing the uploaded image path).

During the call stack backtracking phase, for the call stack corresponding to the i-th suspicious function s_c^i, we start from the top function s_i^0 and sequentially attempt to apply the hooking strategies, determining whether the attack can be bypassed without causing program exceptions. If s_i^0 cannot be bypassed, we continue the same judgment for $s_i^1, s_i^2, \ldots$ until we find the first function s_i^q that can successfully bypass the image processing attack. This process can be formally represented as finding the smallest positive integer q that satisfies:

$$\begin{cases} \boldsymbol{I}_s = \boldsymbol{I}'_s, & \text{if } j = q \\ \boldsymbol{I}_s \neq \boldsymbol{I}'_s, & \text{if } j \in [0, q) \end{cases}, \tag{7}$$

where j is an integer index variable. s_i^q is the lossy function to be localized, and for ease of representation, it will be denoted as f_l in the following text.

This process can also be described by Algorithm 1. The input of this algorithm is the target app and stego image, and the output is f_l. getMethod(·) denotes obtaining the function corresponding to the statements in the suspicious function set S_c, and getStack(·) denotes obtaining the call stack corresponding to the triggered function. The hooking operation Hook($f_l, \boldsymbol{I}_s$) can be defined as:

$$\text{Hook}(f_l, \boldsymbol{I}_s) = \boldsymbol{I}_s, \tag{8}$$

where the hooking operation intercepts the call to the lossy function f_l and applies an appropriate strategy from the set L (see Table II) to bypass the attack, ensuring that the stego image $\boldsymbol{I}_s$ is transmitted without any modification.

In summary, this method systematically identifies and verifies image processing or upload functions that can effectively protect secret information through dynamic monitoring and stack structure analysis, thereby achieving accurate localization and bypassing of lossy functions.

Algorithm 1 Localization Algorithm of Lossy Functions

Input: Target app, $\boldsymbol{I}_s$;
Output: Lossy functions f_l;

```
Initialize S_v, S_c, S_i;
Decompile the app to obtain low-level language;
Convert the low-level language to high-level language to obtain S_h;
Filter S_h using semantic features P to obtain the characteristic statements set S_v;
Filter S_v using syntactic features R to obtain the suspicious statements set S_c;
for all i = {0, 1, ...} do
    S_i = getMethod(S_c, i); // getMethod(S_c, i) returns the function corresponding to the i-th suspicious statement.
    for all q = {0, 1, ...} do
        s_i = S_i[q];
        s_i^q = getStack(s_i, q); // getStack(s_i, q) returns the call stack of the triggered function s_t^i.
        f_l = s_i^q;
        if Hook(f_l, I_s)==I_s then
            return f_l;
        end if
    end for
end for
return None;
```

E. Covert Communication Process

In the covert communication phase, the sender first inputs the secret information m and the cover image $\boldsymbol{I}_c$ into the specified steganographic algorithm to obtain the stego image $\boldsymbol{I}_s$, which can be formally represented as:

$$\boldsymbol{I}_s = \text{Hide}(\boldsymbol{I}_c, m), \tag{9}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

where $\text{Hide}(\cdot,\cdot)$ is the information hiding function.

Subsequently, based on the obtained lossy function f_l, we intercept it, modify the image processing pipeline, and upload the stego image $\boldsymbol{I}_s$ to the social-platform server, which can be formally expressed as:

$$S \leftarrow \text{Upload}(\text{Hook}(f_l, \boldsymbol{I}_s)), \tag{10}$$

where S denotes the server, and $\text{Upload}(\cdot)$ denotes the image upload function.

However, not all stego images have higher extraction accuracy before being attacked than after; for details, refer to Experiment V-D.2. Therefore, the following adaptive bypassing strategy can be employed to handle the stego image:

$$S \leftarrow \begin{cases} \text{Upload}(\text{Hook}(f_l, \boldsymbol{I}_s)), & e_a > e_u \\ \text{Upload}(\boldsymbol{I}_s), & e_a \le e_u \end{cases}, \tag{11}$$

where e_a represents the extraction accuracy of the stego image after being attacked, and e_u represents the extraction accuracy of the stego image without being attacked. Specifically, this strategy first pre-attacks the stego image and detects the extraction accuracy before and after the attack. If the extraction accuracy decreases, the bypass strategy is adopted. If the accuracy improves, the social app's conventional image processing flow is executed when transmitting the stego image.

After the stego image is published on the social platform, the receiver can retrieve the sender's images to obtain the stego image and use the corresponding steganographic method to completely extract the secret information sent by the sender.

$$m' = \text{Ext}(\boldsymbol{I}'_s), \tag{12}$$

where m' denotes the extracted secret information, and $\text{Ext}(\cdot)$ is the extraction function corresponding to $\text{Hide}(\cdot,\cdot)$.

It is worth noting that whether m' equals m depends on the specific steganographic algorithm. In the absence of attacks, if the steganographic algorithm can ensure the complete extraction of the secret information from the stego image, then $m' = m$; otherwise, $m' \neq m$. This means that HookSteg can only guarantee the lossless transmission of secret information, provided that the secret information itself is extractable in the original channel. In other words, if the information cannot be completely extracted from the stego image even in the absence of attacks, then HookSteg likewise cannot ensure the complete extraction of the information.

IV. Performance Analysis

This section analyzes the proposed method from three aspects: losslessness, time complexity, and memory (space) complexity, demonstrating the effectiveness of the proposed method in principle.

A. Losslessness Analysis

Theorem 1: Lossless Channel Theorem. Under the HookSteg, by intercepting the lossy function f_l, the sender can ensure that the secret information m' extracted after the stego image $\boldsymbol{I}_s$ is processed by the social platform is completely identical to the original information m.

Proof: Let the original image be $\boldsymbol{I}_c$, the secret information be m, the hiding algorithm be $\text{Hide}(\boldsymbol{I}_c, m) \rightarrow \boldsymbol{I}_s$, and the extraction algorithm be $\text{Ext}(\boldsymbol{I}'_s) \rightarrow m$.

(1) **Sender-side Process.** Define the image processing pipeline as a function composition:

$$F = f_n \circ f_{n-1} \circ \cdots \circ f_0, \tag{13}$$

where "$\circ$" denotes function composition. $f_0, \ldots, f_n$ are lossless operations such as network transmission and protocol encapsulation apart from f_l.

Then the actual processing pipeline becomes:

$$F' = f_n \circ \cdots \circ f_{l+1} \circ \text{Hook}(f_l, \cdot) \circ f_{l-1} \circ \cdots \circ f_0. \tag{14}$$

Since $\text{Hook}(f_l, \boldsymbol{I}_s)$ directly returns the input for the stego image, and all functions after f_l operate on the original stego image, therefore:

$$\begin{aligned} F'(\boldsymbol{I}_s) = \; & f_n(\boldsymbol{I}_s) \circ \cdots \circ f_{l+1}(\boldsymbol{I}_s) \circ \text{Hook}(f_l, \boldsymbol{I}_s) \\ & \circ f_{l-1}(\boldsymbol{I}_s) \circ \cdots \circ f_0(\boldsymbol{I}_s). \end{aligned} \tag{15}$$

(2) **Receiver-side Process.** Define the deterministic encoding function $D(\cdot)$ (such as format encapsulation, network coding, etc.) that does not alter image content. The actual received image is:

$$\boldsymbol{I}_r = D(F'(\boldsymbol{I}_s)). \tag{16}$$

The extraction algorithm $\text{Ext}(\cdot)$ satisfies the robustness condition:

$$\forall \boldsymbol{I}'_s, \text{if } \text{SSIM}(\boldsymbol{I}'_s, \boldsymbol{I}_s) > \theta, \text{then } \text{Ext}(\boldsymbol{I}'_s) = \text{Ext}(\boldsymbol{I}_s), \tag{17}$$

where θ is the image similarity threshold, and SSIM (Structural Similarity Index Measure) is a metric for assessing the similarity between two images. When the SSIM value is 1, the two images can be considered identical.

(3) **Losslessness Derivation.** From Eq. 17, the attack from the lossy function f_l is not included, therefore:

$$F'(\boldsymbol{I}_s) = \boldsymbol{I}_s. \tag{18}$$

For common social platforms, $D(\cdot)$ does not alter image content, hence:

$$\text{SSIM}(\boldsymbol{I}_r, \boldsymbol{I}_s) = 1 > \theta. \tag{19}$$

Therefore:

$$m' = \text{Ext}(\boldsymbol{I}_r) = \text{Ext}(\boldsymbol{I}_s) = m. \tag{20}$$

HookSteg does not modify any pixel values or image metadata. It only redirects the control flow of the application. Consequently, it introduces no additional artifacts to the stego image. The transmitted image is bitwise identical to the original stego image generated by the steganographic algorithm.

B. Time Complexity Analysis

Theorem 2: Processing Time Complexity Theorem. For any stego image $\boldsymbol{I}_s$, the processing time T_{hook} of the HookSteg channel is constant. In contrast, the processing time $T_{\text{conventional}}$ of the conventional channel is linearly related to the total number of image pixels n, namely:

$$T_{\text{hook}} = O(1) < T_{\text{conventional}} = O(n). \tag{21}$$

Proof: (1) **Time Complexity of Conventional Channel.** A conventional image processing pipeline includes operations such as scaling and compression, and its time complexity is:

$$T_{\text{conventional}} = \sum_{i=0}^{k} T_{f_i}(\boldsymbol{I}_s) = O(n), \tag{22}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

where n is the total number of pixels. Scaling and compression algorithms need to traverse all pixels, hence $T_{f_i} = O(n)$.

(2)**Time Complexity of HookSteg Channel.** HookSteg bypasses the lossy function f_l through kernel-level hooking, and the actual execution sequence is:

$$P_{\text{hook}}(\boldsymbol{I}_s) = f_k(\boldsymbol{I}_s) \circ \cdots \circ f_{l+1}(\boldsymbol{I}_s) \circ \text{Hook}(f_l, \boldsymbol{I}_s) \circ f_{l-1}(\boldsymbol{I}_s) \circ \cdots \circ f_0(\boldsymbol{I}_s), \tag{23}$$

where $f_0, \ldots, f_k$ are lossless operations such as network transmission and protocol encapsulation, with a time complexity of $O(1)$.

The hooking framework's own overhead T_{overhead} includes conditional checking, control flow transfer, and context restoration, all of which are constant-time operations.

Therefore:

$$T_{\text{hook}} = T_{\text{overhead}} + \sum_{j=0}^{k} T_{f_j}(\boldsymbol{I}_s) = O(1) + O(1) = O(1). \tag{24}$$

(3) **Quantification of Time Advantage.** The ratio of processing time between the two channels is:

$$\eta = \frac{T_{\text{conventional}}}{T_{\text{hook}}} = \frac{O(n)}{O(1)} = \Omega(n), \tag{25}$$

where $\Omega(n)$ denotes the asymptotic lower bound, indicating that the time advantage over conventional methods grows at least linearly with the image size, highlighting its significant superiority in large-scale image processing.

By circumventing pixel-level processing with $O(n)$ complexity, HookSteg reduces the processing time of stego images to a constant level, achieving theoretically optimal efficiency.

C. Memory Complexity Analysis

To evaluate the practical feasibility of HookSteg, we analyze its memory complexity in this subsection. The memory overhead of HookSteg primarily originates from three sources: (i) the hooking framework itself, (ii) the dynamically generated hook stubs and trampolines, and (iii) the temporary storage of call stack information during the preprocessing phase.

Theorem 3: Memory Complexity Theorem. For any stego image I_s, the memory consumption M_{hook} of the HookSteg channel is constant and independent of the image size n (where n) is the total number of pixels). In contrast, a conventional image processing channel has a memory complexity of $O(n)$.

Proof: (1) **Memory Complexity of Conventional Channel.** A conventional image processing pipeline involves operations such as decompression, scaling, and recompression. These operations typically require the image to be fully or partially loaded into memory. For an image with n pixels, the memory footprint is at least proportional to n. Formally,

$$M_{\text{conventional}} = O(n). \tag{26}$$

(2) **Memory Complexity of HookSteg Channel.** HookSteg bypasses the lossy function f_l by intercepting its call at the kernel or user-space level. The hooking mechanism (e.g., inline hooking or function detouring) introduces:

- A fixed-size jump instruction (typically 5–14 bytes on ARM/ARM64).
- A trampoline function that restores the original control flow (constant size).
- A small hook context structure, also constant.

All of these overheads are independent of the image pixel count n. Additionally, the temporary call stack captured during the preprocessing phase (Algorithm 1) stores only function addresses and return addresses, whose number is bounded by the maximum call stack depth (a constant). Therefore,

$$M_{\text{hook}} = O(1). \tag{27}$$

(3) **Quantification of Memory Advantage.** The memory ratio between the conventional channel and the HookSteg channel is

$$\mu = \frac{M_{\text{conventional}}}{M_{\text{hook}}} = \frac{O(n)}{O(1)} = \Omega(n), \tag{28}$$

which grows at least linearly with the image size. For high-resolution images, the memory saving is several orders of magnitude.

This analysis demonstrates that HookSteg not only reduces processing time to constant but also eliminates the memory footprint required for pixel-level transformations, making it highly practical for resource-constrained mobile devices.

V. Experiments

In this section, we conduct experiments to evaluate the performance of HookSteg. The experiments primarily include the experimental setup, identification of attack nodes, localization efficiency of lossy functions, robustness, and transmission efficiency.

A. Experimental Setup

During the experiment, we use a One Plus Ace2 as the real device and validate HookSteg in an actual network environment. The device runs the Android 13 operating system. To enable in-depth testing and analysis of the proposed method, we employ the hooking tool Frida within a Python environment, utilize the Objection tool for real-time monitoring of function execution, and adopt Reqable as the network data analysis tool. Detailed experimental environment information is provided in Table IV.

For modification-based steganographic methods, the BossBase v1.01 [46] dataset is adopted, with 1,000 images used for each method. Their resolutions are uniformly adjusted to $5,000 \times 5,000$ pixels to test large-capacity scenarios, and they are saved as JPEG images with quality factors (Q_f) of 75 and 85, respectively. For generation-based steganographic methods, 1,000 images are also used, following the original experimental settings of each method and adopting their corresponding publicly available pre-trained models. The specific parameter configurations are shown in Table V.

B. Identification of Attack Nodes

To evaluate the performance of HookSteg in identifying attack nodes, we select the top 50 apps from China's largest third-party mobile app store, "Yingyongbao", and filter out 21 apps that can interact through images as experimental subjects. In addition, we select 7 international mobile apps accessible in China from the Google Play Store. The apps's information and

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

TABLE III

IDENTIFICATION OF ATTACK NODES. (THE TABLE DATA IS AS OF JANUARY 1, 2026. THE SAME APPLIES TO THE FOLLOWING)

App name	Download count	Package name	Version	Type	Validation method	Client-side only	Server-side involved	No processing
Facebook	10B+	com.facebook.katana	501.0.0.61.70	Social	○	-	-	-
X	1B+	com.twitter.android	10.81.0-release.0	Social	○	-	-	-
Pinterest	1B+	com.pinterest	13.6.0	Social	●	×	✓	×
Reddit	100M+	com.reddit.frontpage	2025.07.1	Social	●	×	✓	×
Tumblr	100M+	com.tumblr	38.3.0.110	Image	●	×	✓	×
Quora	50M+	com.quora.android	3.2.30	Q&A	●	×	✓	×
Kutumb	50M+	com.kutumb.android	7.1.7	Social	●	✓	×	×
Medium	10M+	com.medium.reader	4.5.1253348	Social	●	×	✓	×
Hiferr	1M+	com.hiferr	1.0.37	Social	●	✓	×	×
QQ	9B+	com.tencent.mobileqq	9.1.50	Social	○	-	-	-
WeChat	8B+	com.tencent.mm	8.0.56	Social	○	-	-	-
Tencent Video	5B+	com.tencent.qqlive	9.01.25.30019	Video	●	×	✓	×
Taobao	4B+	com.taobao.taobao	10.49.10	Shopping	○	-	-	-
Baidu	3B+	com.baidu.searchbox	15.0.0.10	Search	●	×	×	✓
KuGou Music	2B+	com.kugou.android	20.1.5	Music	●	×	✓	×
Pinduoduo	2B+	com.xunmeng.pinduoduo	7.48.0	Shopping	○	×	✓	×
Gaode Map	2B+	com.autonavi.minimap	15.16.0.2075	Navigation	●	×	✓	×
Weibo	2B+	com.sina.weibo	15.2.2	Social	●	×	✓	×
QQ Music	2B+	com.tencent.qqmusic	14.1.1.8	Music	●	×	✓	×
iQiyi	2B+	com.qiyi.video	16.5.2	Video	●	✓	×	×
Toutiao	2B+	com.ss.android.article.news	12.3.0	News	●	✓	×	×
Youku	2B+	com.youku.phone	11.1.24	Video	●	✓	×	×
Baidu Map	2B+	com.baidu.BaiduMap	20.14.30	Navigation	●	✓	×	×
Meituan	1B+	com.sankuai.meituan	12.35.203	Food Delivery	○	-	-	-
58 Tongcheng	540M+	com.wuba	13.28.3	Trading	●	×	✓	×
Ximalaya	430M+	com.ximalaya.ting.android	9.3.40.3	Audiobook	●	×	×	✓
QQ Reader	420M+	com.qq.reader	8.27.7.888	Reading	◑	✓	×	×
NetEase News	410M+	com.netease.newsreader.activity	113.9	News	●	×	×	✓
Dianping	390M+	com.dianping.v1	11.40.13	Reviews	●	×	✓	×
Ctrip	380M+	ctrip.android.view	8.78.6	Travel	◑	×	✓	×

TABLE IV

EXPERIMENTAL SETUP

No.	Software Name	Version	Remarks
1	Python	3.8.0	Base environment
2	Frida	16.5.9	Hooking tool
3	Reqable	2.30.3	Network traffic analysis tool
4	Objection	1.11.0	Memory exploration

TABLE V

METHODS COMPARED AND THEIR TYPES

Method Name	Year	Format	Type	Principle Type	Resolution
J-UNIWARD [47]	2014	JPEG	Classical	Modification-based	5,000×5,000
DMAS [32]	2018	JPEG	Robust	Modification-based	5,000×5,000
JCRISBE [13]	2021	JPEG	Robust	Modification-based	5,000×5,000
SCRIPT [30]	2024	JPEG	Robust	Modification-based	5,000×5,000
ROAST [12]	2024	JPEG	Robust	Modification-based	5,000×5,000
StegaDDPM [45]	2023	TIFF	Classical	Generation-based	256×256
ERGI [25]	2024	PNG	Robust	Generation-based	512×512
PARIS [23]	2024	JPEG	Robust	Generation-based	64×64
GSD [28]	2025	PNG	Classical	Generation-based	64×64
TransStego [43]	2025	JPEG	Robust	Generation-based	400×400

experimental results are shown in Table III. In the experiment, we use MitM attacks (denoted by "●") and eBPF (denoted by "◑") to identify nodes where images are attacked. When the former is unavailable, the latter is used as a supplementary means. When the nodes where images are attacked cannot be identified, it is denoted by "○". The experimental results also classify the attack nodes of the images, including client-only attacks, server-involved attacks, and neither client nor server attacks.

The experimental results show that different apps have different approaches regarding the nodes where images are attacked. From a statistical perspective, the MitM attack method is widely used in 70.00% of social apps, indicating that this method has high applicability and effectiveness. As a supplement, the eBPF method, together with the MitM attack method, jointly verifies 76.66% of the apps. Therefore, through these two methods, we can accurately identify the attack nodes of most apps.

From the perspective of the nodes where images are attacked, we observe significant differences in image processing strategies across different types of apps. For instance, apps like iQiyi and QQ Reader tend to preserve high-quality original images with superior visual effects, providing greater operational space for implementing lossless steganography. In contrast, apps such as NetEase News and Dianping prioritize storage costs and user experience, often processing images before storing them on servers. Meanwhile, apps like Kutumb and Youku handle image processing exclusively on the client side. In the following experiments, we validate HookSteg's performance using Kutumb, Hiferr, Youku, and Kehua as test cases.

This experiment validates the applicability of HookSteg for establishing lossless channels and reveals the differences in image processing strategies among different types of apps. These findings provide References for future research on image processing technologies and data protection.

C. Localization Efficiency of Lossy Functions

To verify the effectiveness of HookSteg in locating lossy functions, we conduct experiments on the apps listed in Table III that allow attack bypass via HookSteg, including iQiyi, Toutiao, Youku, Baidu Map, QQ Reader, Kutumb, and Hiferr. The experiment precisely locates the lossy functions, and the total number of retrieved, filtered, and extracted suspicious functions for each app, along with the efficiency

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

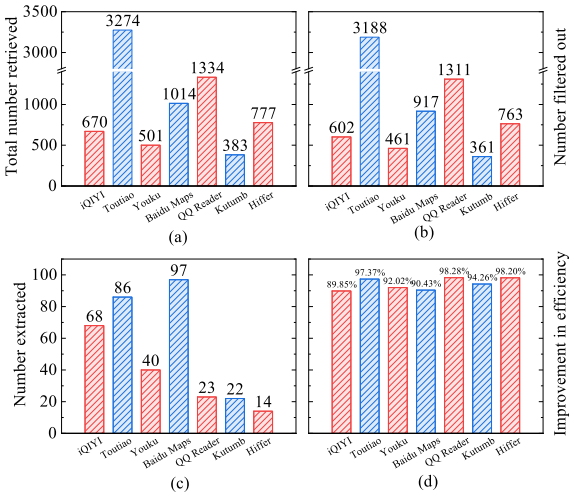


Fig. 6. Experiment on extraction efficiency of lossy functions.

improvement compared to manual analysis, are sequentially presented in Figs. 6 (a), (b), (c) and (d).

It can be seen from the experimental results that HookSteg performs excellently in locating the lossy functions, with an average efficiency improvement of 94.34%. This indicates that the method has a significant effect in reducing unnecessary retrieval volume, thereby improving overall retrieval efficiency. Furthermore, the lossy functions for all apps are found within the extracted suspicious statements, indicating that the method can effectively identify lossy functions, ensuring the accuracy and reliability of the extracting results. The application of this method is expected to play an important role in practical retrieval tasks, greatly enhancing the efficiency of constructing lossless channels.

D. Robustness Experiment

Regarding the selection of comparison methods, this paper focuses on two categories: modification-based steganography and generation-based steganography. Although these two types of methods offer higher embedding capacity, they struggle to achieve lossless recovery under hybrid attacks. To validate the robustness of the proposed method under high-capacity transmission, the aforementioned methods are selected for experimentation. Mapping-based steganography, while exhibiting strong robustness, suffers from excessively low embedding capacity and fails to meet the high-capacity transmission requirements of social platforms. Therefore, it is not included in the comparison.

The real-world experimental scenarios are: Youku, Kehua, Hiferr, and Kutumb. Their respective quality factors for image processing are 80, 60, 100, and 80. The experiments are conducted on conventional channels and the HookSteg channel constructed based on our proposed method.

1) Robustness Experiment for Modification-Based Steganography: To verify the robustness of modification-based steganographic methods in real-world scenarios, we conduct experiments under two scenarios. The first condition uses images whose compression quality factors match those observed from the actual platforms. The second condition uses images whose quality factors do not match those of the platforms. The experimental results are shown in Table VI

TABLE VI

ROBUSTNESS EXPERIMENT OF MODIFICATION-BASED STEGANOGRAPHY UNDER QUALITY FACTORS MATCHED TO REAL-WORLD SCENARIOS

App	Method	Stego↑	Conventional channel		HookSteg channel	
		(No attack)	Attacked↑	Change↑	Attacked↑	Change↑
Youku	J-UNIWARD [47]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
	DMAS [32]	99.99%	50.31%	-49.68%	99.99%	**0.00%**
	SCRIPT [30]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
	ROAST [12]	99.11%	49.92%	-49.19%	99.11%	**0.00%**
	JCRISBE [13]	100.00%	50.02%	-49.98%	100.00%	**0.00%**
Kehua	J-UNIWARD [47]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
	DMAS [32]	99.99%	50.19%	-49.80%	99.99%	**0.00%**
	SCRIPT [30]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
	ROAST [12]	99.99%	50.17%	-49.82%	99.99%	**0.00%**
	JCRISBE [13]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
Hiferr	J-UNIWARD [47]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
	DMAS [32]	50.26%	49.96%	-0.30%	50.26%	**0.00%**
	SCRIPT [30]	-	-	-	-	-
	ROAST [12]	50.10%	49.93%	-0.17%	50.10%	**0.00%**
	JCRISBE [13]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
Kutumb	J-UNIWARD [47]	100.00%	50.02%	-49.98%	100.00%	**0.00%**
	DMAS [32]	99.99%	50.04%	-49.95%	99.99%	**0.00%**
	SCRIPT [30]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
	ROAST [12]	98.68%	49.99%	-48.69%	98.68%	**0.00%**
	JCRISBE [13]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
Mean (All Apps)		94.64%	50.03%	-44.61%	94.64%	**0.00%**

and Table VII, where the data in the last three columns represent the extraction accuracy of stego image under no attack, under attack, and the change after attack, respectively. When the change in extraction accuracy is non-negative, it indicates that the method effectively resists the attack without decreasing extraction accuracy. Such values are highlighted in bold. It is worth noting that the Hiferr scenario uses a quality factor of 100 when compressing images. Under this condition, SCRIPT fails to generate stego images, making it impossible to calculate its extraction accuracy. The corresponding result is denoted by "-".

When using images whose compression quality factors match those observed from the actual platforms, Table VI shows that after being transmitted through conventional channels, the extraction accuracy of stego images generated by J-UNIWARD [47], DMAS [32], SCRIPT [30], ROAST [12], and JCRISBE [13] is around 50%. This result is comparable to random guessing, making it impossible to extract effective secret information. However, after the stego images of J-UNIWARD [47], DMAS [32], SCRIPT [30], ROAST [12], and JCRISBE [13] are transmitted via HookSteg, their extraction accuracy remains consistent with that under no attack. Compared to transmission over conventional channels, the extraction accuracy improves by an average of 44.61%.

When using images whose compression quality factors do not match those observed from the actual platforms, stego images with quality factors of 75 and 85 are selected for the experiment. The experimental results are similar to those obtained when the attack parameters are known. Table VI shows that after transmission over conventional channels, the extraction accuracy of J-UNIWARD [47], DMAS [32], SCRIPT [30], ROAST [12], and JCRISBE [13] is also around 50%. After using HookSteg, the extraction accuracy of these methods is significantly improved, with an average increase of 49.72%.

By conducting experiments on J-UNIWARD [47], DMAS [32], SCRIPT [30], ROAST [12], and JCRISBE [13], this subsection verifies the robustness of modification-based steganographic methods in two scenarios. The experimental results show that, under the hybrid attacks of social

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

TABLE VII

ROBUSTNESS EXPERIMENT OF MODIFICATION-BASED STEGANOGRAPHY UNDER QUALITY FACTORS MISMATCHED TO REAL-WORLD SCENARIOS

Q_f	App	Method	Stego (No attack) ↑	Conventional channel		HookSteg channel	
				Attacked ↑	Change ↑	Attacked ↑	Change ↑
75	Youku	J-UNIWARD [47]	100.00%	50.71%	-49.29%	100.00%	**0.00%**
		DMAS [32]	99.99%	50.28%	-49.71%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	50.29%	-49.71%	100.00%	**0.00%**
		ROAST [12]	99.82%	50.75%	-49.07%	99.82%	**0.00%**
		JCRISBE [13]	100.00%	49.97%	-50.03%	100.00%	**0.00%**
	Kehua	J-UNIWARD [47]	100.00%	49.95%	-50.05%	100.00%	**0.00%**
		DMAS [32]	99.99%	49.99%	-50.00%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	50.67%	-49.33%	100.00%	**0.00%**
		ROAST [12]	99.91%	49.43%	-50.48%	99.91%	**0.00%**
		JCRISBE [13]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
	Hiferr	J-UNIWARD [47]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
		DMAS [32]	99.99%	50.06%	-49.93%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	49.96%	-50.04%	100.00%	**0.00%**
		ROAST [12]	99.84%	49.92%	-49.92%	99.84%	**0.00%**
		JCRISBE [13]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
	Kutumb	J-UNIWARD [47]	100.00%	50.01%	-49.99%	100.00%	**0.00%**
		DMAS [32]	99.99%	50.22%	-49.77%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	49.97%	-50.03%	100.00%	**0.00%**
		ROAST [12]	99.89%	50.40%	-49.49%	99.89%	**0.00%**
		JCRISBE [13]	100.00%	49.99%	-50.01%	100.00%	**0.00%**
85	Youku	J-UNIWARD [47]	100.00%	49.97%	-50.03%	100.00%	**0.00%**
		DMAS [32]	99.99%	49.98%	-50.01%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	50.35%	-49.65%	100.00%	**0.00%**
		ROAST [12]	98.22%	50.13%	-48.09%	98.22%	**0.00%**
		JCRISBE [13]	100.00%	50.03%	-49.97%	100.00%	**0.00%**
	Kehua	J-UNIWARD [47]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
		DMAS [32]	99.99%	50.12%	-49.87%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	49.31%	-50.69%	100.00%	**0.00%**
		ROAST [12]	98.30%	49.65%	-48.65%	98.30%	**0.00%**
		JCRISBE [13]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
	Hiferr	J-UNIWARD [47]	100.00%	49.98%	-50.02%	100.00%	**0.00%**
		DMAS [32]	99.99%	50.01%	-49.98%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	49.91%	-50.09%	100.00%	**0.00%**
		ROAST [12]	98.28%	49.87%	-48.41%	98.28%	**0.00%**
		JCRISBE [13]	100.00%	50.00%	-50.00%	100.00%	**0.00%**
	Kutumb	J-UNIWARD [47]	100.00%	50.02%	-49.98%	100.00%	**0.00%**
		DMAS [32]	99.99%	50.04%	-49.95%	99.99%	**0.00%**
		SCRIPT [30]	100.00%	49.72%	-50.28%	100.00%	**0.00%**
		ROAST [12]	98.97%	50.24%	-48.73%	98.97%	**0.00%**
		JCRISBE [13]	100.00%	50.02%	-49.98%	100.00%	**0.00%**
Mean (Q_f=75)			99.97%	50.13%	-49.84%	99.97%	**0.00%**
Mean (Q_f=85)			99.69%	49.97%	-49.72%	99.69%	**0.00%**

platforms, the extraction accuracy of both non-robust and robust modification-based steganographic methods significantly decreases. However, when the proposed HookSteg is used, the extraction accuracy is significantly improved, with an average increase of 48.06%.

2) Robustness Experiment for Generation-Based Steganography: To verify the effectiveness of HookSteg for generation-based steganographic methods, this subsection selects five representative schemes [23], [25], [28], [43], [45] for experimental evaluation. Relevant results are shown in Table VIII.

Experimental data show that the extraction accuracy of StegaDDPM [45] and GSD [28] after being attacked is around 50%. The extraction accuracy of TransStego [43], ERGI [25], and PARIS [23] is above 80%. The experimental results indicate that the non-robust generation-based steganographic methods StegaDDPM [45] and GSD [28], similar to modification-based steganographic methods, are also unable to resist attacks in real lossy environments. In contrast, the robust generation-based steganographic methods TransStego [43], ERGI [25], and PARIS [23] can resist attacks over lossy channels to varying degrees.

Furthermore, for TransStego [43] transmitted via the Hiferr scenario, the extraction accuracy after being attacked is actually higher than that of the original stego image. This is because JPEG compression, even at quality factor 100, introduces slight quantization errors. This "mild perturbation" might correct certain distribution biases or minor errors introduced during the training or generation of the generation-based steganography model. This allows the extractor to work on data closer to the training conditions, thereby increasing the accuracy. According to HookSteg's autonomous selection strategy in the image processing flow, higher extraction accuracy is ensured. When TransStego's stego images are transmitted via the Hiferr scenario, they still follow the conventional processing flow. This experiment demonstrates that HookSteg significantly enhances the robustness of generation-based steganographic methods, with an average improvement of 25.19%, which also verifies the effectiveness of HookSteg for these methods.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII

ROBUSTNESS EXPERIMENT OF GENERATION-BASED STEGANOGRAPHIC METHODS

App	Method	Stego↑ (No attack)	Conventional channel		HookSteg channel	
			Attacked↑	Change↑	Attacked↑	Change↑
Youku	StegaDDPM [45]	100.00%	50.53%	-49.47%	100.00%	**0.00%**
	ERGI [25]	99.03%	91.66%	-7.37%	99.03%	**0.00%**
	PARIS [23]	98.44%	90.10%	-8.34%	98.44%	**0.00%**
	GSD [28]	99.99%	50.03%	-49.96%	99.99%	**0.00%**
	TransStego [43]	98.80%	98.30%	-0.50%	98.80%	**0.00%**
Kehua	StegaDDPM [45]	100.00%	50.42%	-49.58%	100.00%	**0.00%**
	ERGI [25]	99.03%	88.06%	-10.97%	99.03%	**0.00%**
	PARIS [23]	98.44%	87.50%	-10.94%	98.44%	**0.00%**
	GSD [28]	99.99%	50.07%	-49.92%	99.99%	**0.00%**
	TransStego [43]	98.80%	97.60%	-1.20%	98.80%	**0.00%**
Hiferr	StegaDDPM [45]	100.00%	50.75%	-49.25%	100.00%	**0.00%**
	ERGI [25]	99.03%	85.31%	-13.72%	99.03%	**0.00%**
	PARIS [23]	98.44%	83.85%	-14.59%	98.44%	**0.00%**
	GSD [28]	99.99%	49.95%	-50.04%	99.99%	**0.00%**
	TransStego [43]	98.80%	99.20%	+0.40%	99.20%	**+0.40%**
Kutumb	StegaDDPM [45]	100.00%	50.28%	-49.72%	100.00%	**0.00%**
	ERGI [25]	99.03%	80.92%	-18.11%	99.03%	**0.00%**
	PARIS [23]	98.44%	84.11%	-14.33%	98.44%	**0.00%**
	GSD [28]	99.99%	50.11%	-49.88%	99.99%	**0.00%**
	TransStego [43]	98.80%	92.90%	-5.90%	98.80%	**0.00%**
Mean (All Apps)		99.25%	74.08%	-25.17%	99.27%	**+0.02%**

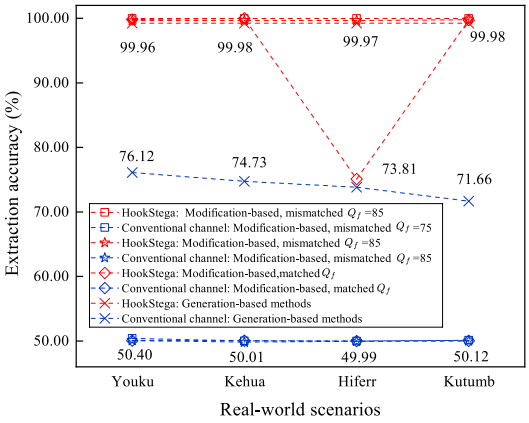


Fig. 7. Extraction accuracy of generation-based steganographic methods.

Fig. 7 further visualizes the above results in the form of a line chart. The horizontal axis represents four real-world scenarios, and the vertical axis represents the extraction accuracy. The red dashed line denotes the HookSteg channel, and the blue dashed line denotes the conventional channel. For modification-based steganographic methods, we show the performance of the HookSteg channel under two mismatched quality factor scenarios (Q_f = 85 and Q_f = 75), as well as the robustness performance under the conventional channel. For generation-based steganographic methods, we similarly present a comparison between the HookSteg channel and the conventional channel. It can be observed that the extraction accuracy of all methods under HookSteg is significantly higher than that under the conventional channel. This figure further demonstrates the robustness of HookSteg against real, unknown, and automated channel attacks.

In this subsection, experimental validation is conducted separately for modification-based steganographic methods and generation-based steganographic methods. The experimental results show that after using HookSteg, the extraction accuracy of the two types of methods achieves an average improvement of 48.06% and 25.19%, respectively, with a comprehensive average improvement of 36.63%. These results indicate that

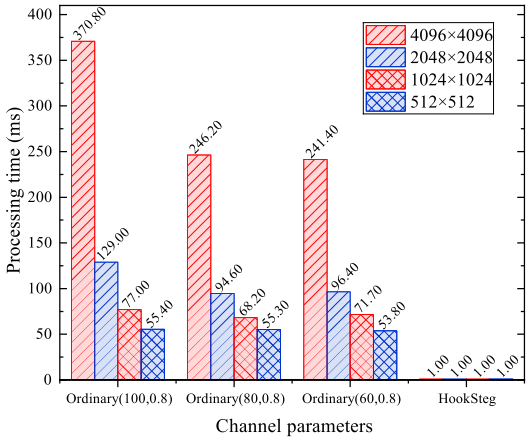


Fig. 8. Comparison experiment of transmission efficiency.



Fig. 9. Representative images used in the experiments.

when stego images are transmitted via real-world scenarios, the proposed method enables lossless transmission of secret information and effectively enhances the robustness of steganographic methods, thereby verifying the excellent robustness performance of HookSteg.

E. Transmission Efficiency

To investigate the impact of HookSteg on the transmission efficiency of stego images, this experiment is conducted with the scaling factor fixed at 0.8 and the compression factor set to 100, 80, and 60, respectively. Four types of images with different resolutions are selected for the experiment: 512×512, $1,024\times1,024$, $2,048\times2,048$, and $4,096\times4,096$ pixels. The experimental results are shown in Fig. 8.

Fig. 8 illustrates the time consumed for processing images across four groups of experiments. Among them, the first three groups of experiments are conducted in a conventional channel environment, with their attack parameters represented in the form of a tuple (quality factor, scaling factor). The fourth group shows the time required for HookSteg to process stego images.

Experimental data show that under the same attack parameters, the processing time for stego images in the conventional channel increases with image size. When the scaling factor and image size are fixed, the processing time is significantly longer for a quality factor of 100, while the times for quality factors 60 and 80 are relatively similar. In the channel established by HookSteg, the time required to process stego images is 1 millisecond, which is unaffected by image size or quality factor. This is because HookSteg bypasses the conventional image processing pipeline, and its processing time is entirely determined by the overhead of the hooking framework itself.

The experimental results indicate that compared to the conventional channel, the channel established by HookSteg

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

introduces no additional transmission burden. Moreover, by avoiding image encoding and decoding processes, it significantly improves image transmission efficiency.

VI. Conclusion

This paper proposes for the first time a lossless steganographic method suitable for lossy channels, aiming to meet the high requirements for information integrity in sensitive data transmission scenarios. The method leverages process hooking to effectively circumvent lossy channel attacks, ensuring that no bit errors occur during the transmission of secret information. Experimental results show that the method enables efficient and lossless covert communication in real-world scenarios, providing strong support for the accurate transmission of high-value sensitive data. Compared with existing steganographic methods, our method effectively breaks the mutual constraints among embedding capacity, security, and robustness, allowing steganography research to focus on the enhancement of security and capacity. This method opens up a new path for the field of digital privacy protection, particularly offering a reliable data security solution for application scenarios with extremely high requirements for information integrity.

VII. Future Work

Although HookSteg can achieve lossless robust steganography under lossy channels, it still has two main limitations. First, HookSteg requires the sender's device to have root privileges to perform process hooking operations, which is difficult to popularize on ordinary user devices. Future work will explore lightweight bypass solutions based on virtualization or sandboxing technologies to serve a broader range of users. Second, HookSteg is applicable only to scenarios where the image is attacked solely on the client side. For applications where lossy processing also exists on the server side, HookSteg cannot guarantee lossless transmission. Future research may incorporate prior knowledge of server-side processing to design targeted robust steganographic coding or error correction mechanisms, enabling partial tolerance of lossy operations on the server side.

Appendix
Representative Image Samples

To provide a clear illustration of the image data used in our experiments, we present representative image samples in Fig. 9. The first image is taken from the BossBase v1.01 dataset [46], which serves as the cover images for modification-based steganographic methods. The subsequent five images are generated by the five generation-based steganographic methods (StegaDDPM [45], ERGI [25], PARIS [23], GSD [28], and TransStego [43]) using their respective publicly available pre-trained models. All images are displayed with their corresponding resolutions explicitly labeled.

References

[1] P. Jiang et al., "Boomerang: Metadata-private messaging under hardware trust," in *Proc. 20th USENIX Symp. Networked Syst. Design Implement. (NSDI)*, Apr. 2023, pp. 877–899.

[2] M. Bai, J. Yang, K. Pang, X. Xu, Z. Yang, and Y. Huang, "Provably robust and secure steganography in asymmetric resource scenario," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2025, pp. 1438–1456.

[3] A. Cheddad, J. Condell, K. Curran, and P. Mc Kevitt, "Digital image steganography: Survey and analysis of current methods," *Signal Process.*, vol. 90, no. 3, pp. 727–752, Mar. 2010.

[4] B. Li, M. Wang, J. Huang, and X. Li, "A new cost function for spatial image steganography," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2014, pp. 4206–4210.

[5] F. Petitcolas, R. Anderson, and M. Kuhn, "Information hiding—A survey," *Proc. IEEE*, vol. 87, no. 7, pp. 1062–1078, Jul. 1999.

[6] T. Pevný, T. Filler, and P. Bas, "Using high-dimensional image models to perform highly undetectable steganography," in *Proc. Int. Workshop Inf. Hiding*, Jun. 2010, pp. 161–177.

[7] D. Huang, W. Luo, M. Liu, W. Tang, and J. Huang, "Steganography embedding cost learning with generative multi-adversarial network," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 15–29, 2024.

[8] Y. Zhang, Y. Ma, Q. Zhang, Y. Pei, and X. Luo, "An image robust batch steganography framework with minimum embedding signs," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 10745–10760, 2025.

[9] W. Lu, J. Zhang, X. Zhao, W. Zhang, and J. Huang, "Secure robust JPEG steganography based on AutoEncoder with adaptive BCH encoding," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 7, pp. 2909–2922, Jul. 2021.

[10] Y. Huang, Z. Liu, Q. Wu, and X. Liu, "Robust image steganography against JPEG compression based on DCT residual modulation," *Signal Process.*, vol. 219, Jun. 2024, Art. no. 109431.

[11] Y. Zhang, X. Luo, Y. Guo, C. Qin, and F. Liu, "Multiple robustness enhancements for image adaptive steganography in lossy channels," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 8, pp. 2750–2764, Aug. 2020.

[12] K. Zeng, K. Chen, W. Zhang, and Y. Wang, "Upward robust steganography based on overflow alleviation," *IEEE Trans. Multimedia*, vol. 26, pp. 299–312, 2024.

[13] Z. Zhao, Q. Guan, H. Zhang, and X. Zhao, "Improving the robustness of adaptive steganographic algorithms based on transport channel matching," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 7, pp. 1843–1856, Jul. 2019.

[14] Q. Liu, X. Xiang, J. Qin, Y. Tan, and Q. Zhang, "A robust coverless steganography scheme using camouflage image," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 4038–4051, Jun. 2022.

[15] L. Zou, J. Li, W. Wan, Q. M. J. Wu, and J. Sun, "Robust coverless image steganography based on neglected coverless image dataset construction," *IEEE Trans. Multimedia*, vol. 25, pp. 5552–5564, 2023.

[16] B. Guo, P. Ping, and F. Xu, "Highly robust and diverse coverless image steganography against passive and active steganalysis," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2771–2787, May 2025.

[17] M. Yang, B. Qi, B. Ma, J. Xu, Y. Xian, and X. Li, "A high-performance region recognition network-enhanced deep CNN for image content perceptual hashing," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 26456–26471, Jul. 2025.

[18] Y. Luo, J. Qin, X. Xiang, and Y. Tan, "Coverless image steganography based on multi-object recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 7, pp. 2779–2791, Jul. 2021.

[19] J. Liu, L. Tan, Z. Zhou, W. Jiang, Y. Li, and P. Chen, "A dynamic YOLO-based sequence-matching model for efficient coverless image steganography," *Comput., Mater. Continua*, vol. 81, no. 2, pp. 3221–3240, Nov. 2024.

[20] Y. Tan, X. Xiang, J. Qin, and Y. Tan, "Robust coverless image steganography based on human pose estimation," *Knowledge-Based Syst.*, vol. 296, Jul. 2024, Art. no. 111873.

[21] L. Meng, X. Jiang, Z. Zhang, Z. Li, and T. Sun, "A robust coverless image steganography based on an end-to-end hash generation model," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 7, pp. 3542–3558, Jul. 2023.

[22] X. Liu, Z. Ma, J. Ma, J. Zhang, G. Schaefer, and H. Fang, "Image disentanglement autoencoder for steganography without embedding," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Jun. 2022, pp. 2293–2302.

[23] Z. Yang, K. Chen, K. Zeng, W. Zhang, and N. Yu, "Provably secure robust image steganography," *IEEE Trans. Multimedia*, vol. 26, pp. 5040–5053, 2024.

[24] W. Su, J. Ni, and Y. Sun, "StegaStyleGAN: Towards generic and practical generative image steganography," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 1, pp. 240–248.

[25] X. Hu, S. Li, Q. Ying, W. Peng, X. Zhang, and Z. Qian, "Establishing robust generative image steganography via popular stable diffusion," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8094–8108, 2024.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.

[26] J. Yu, X. Zhang, Y. Xu, and J. Zhang, "CRoSS: Diffusion model makes controllable, robust and secure image steganography," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Sep. 2023, pp. 80730–80743.
[27] Y. Qi, K. Chen, N. Zhao, Z. Yang, and W. Zhang, "Provably secure image robust steganography via cross-modal error correction," *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 25, pp. 26354–26362, Apr. 2025.
[28] Q. Zhou, P. Wei, Z. Qian, X. Zhang, and S. Li, "Improved generative steganography based on diffusion model," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 7, pp. 6494–6507, Jul. 2025.
[29] W. Alexan, E. Mamdouh, A. Aboshousha, Y. S. Alsahafi, M. Gabr, and K. M. Hosny, "Stegocrypt: A robust tri-stage spatial steganography algorithm using TLM encryption and DNA coding for securing digital images," *IET Image Process.*, vol. 18, no. 13, pp. 4189–4206, Sep. 2024.
[30] K. Zeng, K. Chen, J. Zhang, W. Zhang, and N. Yu, "Toward secure and robust steganography for black-box generated images," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3237–3250, Feb. 2024.
[31] Y. Zhang, X. Luo, C. Yang, D. Ye, and F. Liu, "A JPEG-compression resistant adaptive steganography based on relative relationship between DCT coefficients," in *Proc. 10th Int. Conf. Availability, Rel. Secur.*, Oct. 2015, pp. 461–466.
[32] Y. Zhang, X. Zhu, C. Qin, C. Yang, and X. Luo, "Dither modulation based adaptive steganography resisting JPEG compression and statistic detection," *Multimedia Tools Appl.*, vol. 77, no. 14, pp. 17913–17935, Jul. 2018.
[33] X. Yu, K. Chen, Y. Wang, W. Li, W. Zhang, and N. Yu, "Robust adaptive steganography based on generalized dither modulation and expanded embedding domain," *Signal Process.*, vol. 168, Mar. 2020, Art. no. 107343.
[34] Y. Zhang, X. Luo, X. Zhu, Z. Li, and A. G. Bors, "Enhancing reliability and efficiency for real-time robust adaptive steganography using cyclic redundancy check codes," *J. Real-Time Image Process.*, vol. 17, no. 1, pp. 115–123, Feb. 2020.
[35] M. J. Swain and D. H. Ballard, "Color indexing," *Int. J. Comput. Vis.*, vol. 7, no. 1, pp. 11–32, Nov. 1991.
[36] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, Nov. 2004.
[37] Y. Ma, L. Xu, Q. Zhang, Y. Zhang, X. Xin, and X. Luo, "EIS-OBEA: Enhanced image steganalysis via opposition-based evolutionary algorithm," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3616–3631, Mar. 2025.
[38] Z. Tang, X. Zhang, and S. Zhang, "Robust perceptual image hashing based on ring partition and NMF," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 3, pp. 711–724, Mar. 2014.
[39] I. Goodfellow et al., "Generative adversarial networks," *Commun. ACM*, vol. 63, no. 11, pp. 139–144, Oct. 2020.
[40] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2015, pp. 2256–2265.
[41] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2020, pp. 6840–6851.
[42] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021, pp. 1–22.
[43] C. Xiao, S. Peng, L. Zhang, J. Wang, D. Ding, and J. Zhang, "A transformer-based adversarial network framework for steganography," *Expert Syst. Appl.*, vol. 269, Apr. 2025, Art. no. 126391.
[44] L. Mescheder, A. Geiger, and S. Nowozin, "Which training methods for GANs do actually converge?" in *Proc. Int. Conf. Mach. Learn.*, Jul. 2018, pp. 3481–3490.
[45] Y. Peng, D. Hu, Y. Wang, K. Chen, G. Pei, and W. Zhang, "StegaDDPM: Generative image steganography based on denoising diffusion probabilistic model," in *Proc. ACM Int. Conf. Multimed. (MM)*, Oct. 2023, pp. 7143–7151.
[46] P. Bas, T. Filler, and T. Pevný, "'Break our steganographic system': The ins and outs of organizing boss," in *Proc. Int. Conf. Inf. Hiding (IH)*, May 2011, pp. 59–70.
[47] V. Holub, J. Fridrich, and T. Denemark, "Universal distortion function for steganography in an arbitrary domain," *EURASIP J. Inf. Secur.*, vol. 2014, no. 1, pp. 1–13, Jan. 2014.



Mingliang Zhang received the M.S. degree from Information Engineering University, Zhengzhou, China, in 2022, where he is currently pursuing the Ph.D. degree. His research interests include information hiding and multimedia security.



Yi Zhang received the B.S., M.S., and Ph.D. degrees from the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, China, in 2014, 2017, and 2021, respectively. She is currently a Teacher with the Key Laboratory of Cyberspace Situation Awareness of Henan Province, Zhengzhou, China. Her research interests include information hiding and multimedia security.



Yanmei Liu received the B.S. degree from Zhengzhou University of Aeronautics, Zhengzhou, China, in 2007, and the M.S. degree from Chongqing University, Chongqing, China, in 2011. She is currently pursuing the Ph.D. degree with Information Engineering University, Zhengzhou, China. Her research interests include multimedia security and image robust steganography.



Wentong Fan received the M.S. degree from Information Engineering University, Zhengzhou, China, in 2023, where he is currently pursuing the Ph.D. degree. His research interests include face reenactment and multimedia security.



Xiangyang Luo received the B.S., M.S., and Ph.D. degrees from the State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou, China, in 2001, 2004, and 2010, respectively. He is the author or co-author of more than 150 refereed international journals and conference papers. He is currently a Professor with Henan Key Laboratory of Cyberspace Situation Awareness. His research interests include image steganography and steganalysis techniques.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:13:30 UTC from IEEE Xplore. Restrictions apply.